
How the sync works

What was built, step by step

This document explains, in easy English, what was built for `SYNC_PLAN.md` (Phases 1 to 10). It follows the order of the work. For each part you get:

- **the idea**: what problem it solves;
- **the files**: where the code is;
- **the code, in short**: a small piece or a summary, enough to understand it.

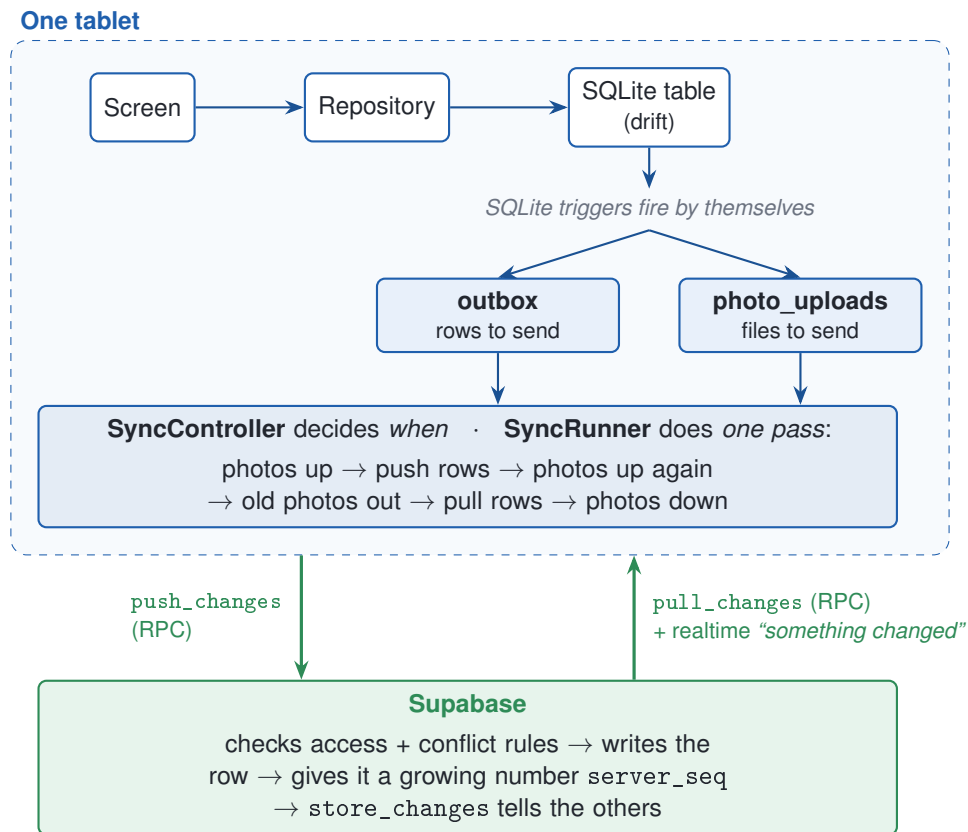
`SYNC_PLAN.md` is the plan and its status notes. This document is the tour of the result.

Contents

The whole picture in one minute	3
1 Make the local database ready for sync (schema v15)	3
1.1 The list of synced tables	3
1.2 Two new columns on every synced table	4
1.3 updated_at is stamped by triggers	4
1.4 Soft deletes	4
1.5 Stock is “derived”: the StockLedger	5
1.6 Real password hashing and a device id	5
2 The outbox (schema v16 and v17)	6
2.1 The outbox table	6
2.2 The outbox triggers (generated)	6
2.3 The “quiet” switch	7
2.4 The pending count and refused changes	7
3 The server (Supabase)	7
3.1 Accounts	7
3.2 Mirror tables (generated)	7
3.3 push_changes: receiving a batch	8
3.4 pull_changes: handing changes out	8
3.5 Photos and join codes	9
3.6 Server tests	9
4 Real login (account + PIN)	9
4.1 Two levels of session	9
4.2 The server behind an interface	9
4.3 Server address and key	10
4.4 Demo mode or account mode	10
5 Sending changes (push)	11
5.1 Two layers	11
5.2 The push loop	11
5.3 When a pass runs	11
6 Receiving changes (pull)	11
6.1 Always push before pull	12
6.2 The pull loop, page by page	12
6.3 Writing received rows: SyncApplier	12
6.4 Live updates (realtime)	13
7 Conflict rules	13
7.1 On the server (in push_changes, see 3.3)	13
7.2 On the device (in SyncApplier, schema v19)	13
7.3 Duplicate names	14
8 Photos (schema v20)	14
8.1 Photo files have their own queue	14
8.2 Photos in the pass	14
9 The data already on a device	15

10 Screens and messages	15
Local schema versions added	16
Tests	16
If you change something later	16
What is left	17

The whole picture in one minute



Five ideas hold everything together

1. **The screen only reads the local database.** It never waits for the network. Offline is normal.
2. **Nothing is really deleted.** A delete sets `deleted_at`, so the delete can travel to the other tablets.
3. **SQLite triggers record every change** into a queue (the outbox). A screen or repository cannot forget to do it.
4. **The server numbers every change** (`server_seq`). Each tablet remembers the last number it received and asks “give me everything after *N*”.
5. **Stock quantity and average cost are not sent.** Each tablet recomputes them from the stock movements, so two tablets always end with the same numbers.

Phase 1 Make the local database ready for sync (schema v15)

1.1 The list of synced tables

Every table must be in one of two lists: sent to the server, or kept on the device.

Files `lib/data/database/sync_tables.dart`

```
static const Set<String> synced = { 'stores', 'categories', 'units', 'items', ... }; // 20
↳ tables
```

```
static const Set<String> local = { 'meta', 'outbox', 'sync_errors', 'photo_uploads' };
```

The order of synced matters: parents come first (a store before its articles, an article before its movements). The tools in `tool/` read this list to generate code, and a test fails if a table is in no list.

1.2 Two new columns on every synced table

To sync, we must know **when** a row changed and **if** it was deleted.

Files `lib/data/database/tables/sync_columns.dart` (a mixin used by every synced table)

```
DateTimeColumn get updatedAt => dateTime().clientDefault(syncStampNow()); // when it last
    ↪ changed (UTC)
DateTimeColumn get deletedAt => dateTime().nullable(); // null = alive
```

Child tables (order lines, attendance sessions, supplier prices, credentials...) also got a `storeId` column, filled from their parent by the migration. The server uses it to check “is this row in your restaurant?” quickly.

1.3 updated_at is stamped by triggers

If one repository forgot to update `updated_at`, that change would never reach the other tablets. A trigger can't forget.

Files `lib/data/database/sync_triggers.drift`

```
CREATE VIEW sync_clock AS SELECT strftime('%Y-%m-%dT%H:%M:%fZ', 'now') AS now; -- UTC "now"

CREATE TRIGGER categories_touch AFTER UPDATE ON categories
  WHEN NEW.updated_at IS OLD.updated_at -- the write didn't set
    ↪ it itself
  AND NOT EXISTS (SELECT 1 FROM meta WHERE "key" = 'syncQuiet') -- and we are not in a
    ↪ quiet write
BEGIN
  UPDATE categories SET updated_at = (SELECT now FROM sync_clock) WHERE rowid = NEW.rowid;
END;
```

Why UTC On Windows, SQLite's `localtime` returns NULL, so the stamps are always UTC.

1.4 Soft deletes

A row that is really removed can't tell the other tablets “I was deleted”.

Files `lib/data/repositories/soft_delete.dart`, used by the catalog, item, supplier, order, credential and calendar repositories.

```
// Before: await _db.delete(_db.items)..where(...).go();
// Now:    SoftDelete(db).item(itemId);
// → sets deleted_at on the article AND on what used to cascade
// (its movements, its supplier links, their price history).
```

ON DELETE CASCADE doesn't fire for a flag, so each cascade is written by hand in `SoftDelete`. Every read query now adds `deletedAt IS NULL`, so deleted rows disappear from the screens.

1.5 Stock is “derived”: the StockLedger

Today each tablet does `quantity = quantity + 5`. If two offline tablets each add 5 kg and we copy the final number, one addition is lost. The fix: never send the quantity. Each tablet rebuilds it from the movements.

Files `lib/data/repositories/stock_ledger.dart`, `lib/data/repositories/movement_repository.dart`

```
class StockLedger {
  Future<bool> rebuildItem(String itemId);           // one article
  Future<Set<String>> rebuildItems(Iterable<String> ids);
}
// Replay, oldest movement first (occurredAt, then id):
// start = item.baselineQuantity / item.baselineAverageCost
// for each movement not already in the baseline:
//   a count (adjustment) → stock = countedQuantity      (absolute at its own time)
//   anything else       → stock += quantity, cost = appliedCostOf(...)
// write item.quantity, item.averageCost, and each movement's unitCost / averageCostAfter
```

Two details:

- **Baseline.** The demo and older installs have stock that doesn't match their movement history. So each article got `baselineQuantity / baselineAverageCost`, and each old movement got an `inBaseline` flag. The replay starts from the baseline.
- **A count is absolute.** Tablet A counts 10 kg while tablet B takes out 3 kg. If B's 3 kg went out *before* the count, the answer is 10 (A saw the shelf after). If they went out *after*, the answer is 7. Adding the stored difference would give 7 both times, which is wrong.

`appliedCostOf` is one function used by both the live path and the replay, so the two can't give different answers.

1.6 Real password hashing and a device id

Files `lib/core/utils/password_hash.dart`, `lib/data/repositories/device_repository.dart`

```
PasswordHash.hash('1234')           // "pbkdf2-sha256$60000$<salt>$<hash>"
PasswordHash.verify('1234', stored) // true / false
DeviceRepository(db).deviceId()      // random id, created once, kept in `meta`
```

The v15 migration rehashed the old passwords, so they still work.

PINs are not hashed In this app the PIN is an identifier shown on the employee cards and used by search. It is not a secret.

Phase 2 The outbox (schema v16 and v17)

2.1 The outbox table

A queue of “rows to send”. **One entry per row**, holding the row as it is now.

Files lib/data/database/tables/outbox.dart

Column	Meaning
id	send order (auto-increment)
changed_table, row_key	which row (busy_dates uses store_id day)
store_id	the restaurant establishment, for the server’s access check
payload	the whole row as JSON, without items.quantity and items.average_cost
queued_at, attempts, last_error	timing and failures

There’s no “delete” entry: a delete is a normal entry whose payload has deleted_at set.

2.2 The outbox triggers (generated)

Files tool/generate_outbox_triggers.py → lib/data/database/outbox_triggers.drift

The Python script reads the latest drift schema dump and writes two triggers per synced table:

```
CREATE TRIGGER items_outbox_update AFTER UPDATE ON items
  WHEN NOT EXISTS (SELECT 1 FROM meta WHERE "key" = 'syncQuiet')
BEGIN
  INSERT INTO outbox (changed_table, row_key, store_id, payload, queued_at)
  SELECT 'items', id, store_id, json_object('id', id, 'name', name, ...), (SELECT now FROM
    ↪ sync_clock)
  FROM items WHERE rowid = NEW.rowid -- reads the row back: always the
    ↪ final version
  ON CONFLICT (changed_table, row_key) DO UPDATE SET -- already queued? replace the payload,
    ↪
    payload = excluded.payload, queued_at = excluded.queued_at, -- but KEEP the id (its
    ↪ place)
    attempts = 0, last_error = NULL;
END;
```

Why keep the id A parent created before its children must still be sent first, even if the parent is edited again later.

2.3 The “quiet” switch

Some writes change synced tables but must **not** be sent: the demo seed, a stock rebuild, and rows received from the server (otherwise they would bounce back).

Files lib/data/repositories/sync_quiet.dart

```
await SyncQuiet.run(db, () async {
  // writes here are NOT queued: meta holds the key 'syncQuiet', and every trigger checks it
});
// The key is added and removed in the SAME transaction → a crash rolls it back too,
// so the queue can never stay switched off.

await SyncQuiet.loud(db, () async { ... }); // inside a quiet write: switch queueing back ON
// (used when a conflict is settled on receipt)
```

2.4 The pending count and refused changes

Files lib/data/repositories/outbox_repository.dart, lib/data/repositories/sync_error_repository.dart, lib/data/database/tables/sync_errors.dart (v17)

```
OutboxRepository(db).pendingCount() // the number on the offline banner (live stream)
OutboxRepository(db).pending(limit: 100) // the next batch, oldest first
OutboxRepository(db).removeIfUnchanged(e) // remove only if not edited while travelling
OutboxRepository(db).reject(e, reason: ...) // move to sync_errors, leave the queue
```

Phase 3 The server (Supabase)

All server code lives in supabase/. It is plain SQL, versioned in git, applied with `supabase db push`. `supabase/README.md` explains it.

3.1 Accounts

Files supabase/migrations/20260929000100_accounts.sql

```
organizations (id, name) one restaurant business
members (user_id, organization_id, role: owner | manager) one organization per login
devices (id, organization_id, name, last_seen_at)
create_organization(name) → the owner's restaurant
register_device(id, name) → ties a tablet to the restaurant
```

3.2 Mirror tables (generated)

Files tool/generate_server_schema.py → supabase/migrations/20260929000200_sync_tables.sql

One Postgres table per synced table, same columns, plus:

```

server_seq          bigint    -- set by a trigger from ONE shared sequence, on every insert/
    ↪ update
updated_by_device text        -- which tablet sent the last change

create trigger items_seq before insert or update on public.items
    for each row execute function private.stamp_server_seq();          -- new.server_seq :=
    ↪ nextval(...)
create trigger items_store_change after insert or update on public.items
    for each row execute function private.note_store_change('store_id'); -- bumps store_changes

```

store_changes (store_id, last_seq) is one tiny table. Tablets listen to it (realtime) instead of to 20 tables.

Security (row level security) Clients can only **read** rows of their own organization (private.can_access_store(store_id)). They can't write any table directly. Every write goes through push_changes, so its rules can't be skipped.

3.3 push_changes: receiving a batch

Files supabase/migrations/20260929000300_push_changes.sql

Each entry is checked in this order, in its own sub-transaction (one bad entry never blocks the others):

#	Check	Result if it fails
1	the device is registered to the caller's organization	device_unknown
2	the table is a synced one	unknown_table
3	the store belongs to the caller; stores: owners only	no_access / owner_only
4	a row can't move to another store	store_changed
5	delete wins : deleted on server, edit arrives	deleted
6	commande status only moves forward	status_backwards / status_closed
7	a paid pay period is final	already_paid
8	otherwise: insert or replace the whole row (last write wins)	accepted + its server_seq

The answer is one line per entry:

```

{"id": 12, "status": "accepted", "seq": 845}
{"id": 13, "status": "rejected", "reason": "deleted"}

```

3.4 pull_changes: handing changes out

```

pull_changes(p_store_id, p_after, p_limit)  -- every row of every table with server_seq >
    ↪ p_after
-- returns { changes: [{seq, table, row}], next_after, has_more }, sorted by server_seq.
-- Deleted rows are included, so tablets learn about deletes.

```

3.5 Photos and join codes

- 20260929000400_photos.sql: a **private** storage bucket photos, one folder per store (<store>/items/<file>, <store>/employees/<file>), with the same access rule.
- 20260929000500_join_codes.sql: create_join_code() (owner only, 8 characters, one use, 7 days), join_organization(code) (the manager joins), my_account(), remove_device(id).

3.6 Server tests

supabase/tests/database/sync.test.sql and accounts.test.sql: **49 pgTAP tests**. Restaurant A can't see restaurant B, push and pull work, every conflict rule, join codes and roles. Run with supabase test db.

Phase 4 Real login (account + PIN)

4.1 Two levels of session

Session	Who	Where	Needed for
Account (Supabase, e-mail + password)	owner or manager	auth_service.dart	syncing
Employee (PIN + 4-digit password)	whoever uses the tablet now	SessionRepository (local)	using the app

The account is signed in once per tablet. After that, employees use their PIN, even offline.

4.2 The server behind an interface

Files lib/services/auth_service.dart

```
abstract interface class AccountBackend {
  // accounts
  Future<AccountUser> signIn({email, password}); Future<AccountUser> signUp({...});
  Future<void> signOut(); Future<void> sendPasswordReset(String email);
  Future<AccountSummary> myAccount();
  Future<String> createOrganization(String name); Future<String> joinOrganization(String code)
    ⇨ ;
  Future<String> createJoinCode();
  Future<void> registerDevice(String id, {name}); Future<List<DeviceInfo>> devices();
  Future<bool> removeDevice(String id);
  // sync
  Future<List<PushResult>> pushChanges(String deviceId, List<Map> changes);
  Future<List<String>> storeIds();
  Future<PullPage> pullChanges(String storeId, {required int after, int limit});
  Stream<LiveSignal> storeChanges(List<String> storeIds); // realtime
  // photos
  Future<void> uploadPhoto(path, bytes, {contentType}); Future<Uint8List?> downloadPhoto(path)
    ⇨ ;
  Future<void> removePhotos(List<String> paths);
}
// SupabaseAccountBackend → the real one
// UnconfiguredAccountBackend → a build with no server: demo only
```

```
// FakeAccountBackend (test/support/) → in memory, for tests
```

Server errors become an `AccountErrorCode` (network, badCredentials, emailTaken, invalidCode, sessionExpired...), and `accountErrorMessage()` turns them into French.

4.3 Server address and key

Files lib/core/config/env.dart

```
static const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
static const supabasePublishableKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');
static bool get hasServer => supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
```

```
flutter run -d windows --dart-define-from-file=config/local.json
```

Keys config/local.json is not committed (config/example.json shows its shape). It holds only the **publishable** key, never the secret one.

4.4 Demo mode or account mode

Files lib/data/device_access.dart, lib/data/repositories/device_access_repository.dart

The device mode is stored in meta, so the router knows it at start, offline, without waiting.

```
class DeviceAccessController extends Notifier<DeviceAccess> {
  startDemo() // the demo, offline, like before
  signIn(email, pw) / signUp(...) // the account login (doesn't change the device yet)
  createRestaurant(...) // owner: organization on server + first store + owner
    ↪ employee
  joinWithCode(code) // manager: join with the owner's code
  finishWithAccount(summary) // tie the device to the account; data comes with first
    ↪ sync
  signOutAccount() // sign out AND wipe the device (lost/handed-over tablet)
}
```

The router guard (lib/app/router.dart) reads deviceAccessSnapshot synchronously:

State	Goes to
no mode yet	welcome page (sign in / create account / «Essayer la démo»)
account, no data	waiting page («Téléchargement des données... N éléments»)
account or demo	PIN login → the app

Screens: lib/features/auth/presentation/pages/ (welcome, sign-up, set-up, forgot, waiting, existing data) and restaurant_account_section.dart in settings (account, join code, devices, sign out).

Phase 5 Sending changes (push)

Files lib/services/sync_service.dart

5.1 Two layers

- **SyncRunner**: does **one** pass. No timers, so tests call it directly.
- **SyncController**: a Riverpod notifier that decides **when** a pass runs, and holds the SyncState that the screens show. It only runs in account mode.

5.2 The push loop

```
Future<SyncRunResult> _push() async {
  for (var batch = 0; batch < maxBatches; batch++) {
    final entries = await outbox.pending(limit: batchSize); // oldest first
    if (entries.isEmpty) break;
    answers = await backend.pushChanges(deviceId, entries); // network error → offline,
    ↪ keep all
    for (final answer in answers) {
      if (answer.accepted) outbox.removeIfUnchanged(entry);
      else if (answer.reason == 'device_unknown') return deviceRemoved; // stop for good
      else outbox.reject(entry, reason: answer.reason);
    }
  }
}
```

- **Accepted** entries leave the queue, **unless the row was edited while it travelled**. Then the newer version stays and goes next time.
- **Refused** entries go to sync_errors and the sync page lists them in French.
- **A removed device or an expired session** stops syncing without retrying, and the sync page says why.

5.3 When a pass runs

```
static const changeDelay = Duration(seconds: 3); // after a local change (groups quick edits)
↪
static const period = Duration(minutes: 5); // safety net while the app is open
static const retryDelays = [5 s, 15 s, 1 min, 5 min]; // after failures in a row
// also: when the app comes back to the front, when the network returns (connectivity_plus),
// when realtime says another tablet changed something, and the "Synchroniser maintenant"
↪ " button.
```

Only one pass runs at a time: syncNow() joins the pass that is already running.

Phase 6 Receiving changes (pull)

6.1 Always push before pull

```
Future<SyncRunResult> run() async {
  photos.upload();           // Phase 8
  final sent = await _push();
  if (sent.outcome != SyncOutcome.done) return sent; // our own changes must be on the
  ↪ server first
  photos.upload(); photos.removeOld();
  final received = await _pull(sent, onReceived);
  photos.downloadMissing();
}
```

6.2 The pull loop, page by page

```
for (final storeId in await backend.storeIds()) { // stores come from the server
  var after = await applier.cursorOf(storeId); // meta key: last server_seq received
  while (true) {
    final page = await backend.pullChanges(storeId, after: after, limit: pageSize);
    received += await applier.apply(storeId, page); // rows + new cursor, one transaction
    onReceived?.call(received); // progress on the waiting screen
    if (!page.hasMore) break;
    after = page.nextAfter;
  }
}
```

If the network drops halfway, the saved cursor means the next pass continues from where it stopped. Nothing is skipped or repeated.

6.3 Writing received rows: SyncApplier

Files lib/data/repositories/sync_applier.dart

For each page, in **one transaction**:

1. **Quiet** (`SyncQuiet.run`): received rows don't go back into the outbox. Since schema v18, the touch triggers are quiet too, so a received row keeps the server's `updated_at`.
2. **Foreign keys are checked at the end of the page** (deferred): a child can arrive before its parent.
3. **A row with an unsent local change is skipped**. The local change is newer; it goes out next, and the server decides.
4. `items.quantity / average_cost` **are never overwritten**. After the page, `StockLedger.rebuildItems(...)` recomputes them for every article that got movements.
5. **Each row has its own savepoint**. A row the local database refuses is logged as `receive_conflict` and the page goes on.
6. **The cursor is saved in the same transaction**.

Values are converted by local column type: booleans to 0/1, and dates to local time with their offset (so an attendance at midnight stays on its day). `updated_at` stays UTC.

6.4 Live updates (realtime)

```
backend.storeChanges(storeIds).listen((_) => schedule(Duration(milliseconds: 500)));
```

The tablet listens to `store_changes`. When another tablet's change reaches the server, a pass starts half a second later. The screens already watch drift streams, so they update by themselves.

The timing bug that was fixed Supabase said “subscribed” *before* the server was really watching, so a change made in that gap was missed. Now the channel asks for `replicationReady: true`, and when the server sends its “ready” system event, the app emits `LiveSignal.ready` and runs a catch-up pass.

Phase 7 Conflict rules

7.1 On the server (in `push_changes`, see 3.3)

Situation	Rule
Two edits of the same row	the last one to reach the server wins (whole row)
One tablet deletes, the other edits	delete wins , the edit is refused (deleted)
Commande status goes back (received → draft)	refused (<code>status_backwards</code> , <code>status_closed</code>)
Same pay period paid twice	first one wins, second refused (<code>already_paid</code>); a retry of the same payment is accepted
Movements, receipts, price history, sessions	add-only: never conflict

Clocks Device clocks never decide who wins. The server's order (`server_seq`) does.

7.2 On the device (in `SyncApplier`, schema v19)

Some “one per” rules exist only on the device. Since v19 they are **partial unique indexes, on live rows only** (`lib/data/database/sync_indexes.drift`):

```
CREATE UNIQUE INDEX attendances_employee_date ON attendances (employee_id, date) WHERE
  ↳ deleted_at IS NULL;
CREATE UNIQUE INDEX supplier_prices_pair ON supplier_prices (item_id, supplier_id) WHERE
  ↳ deleted_at IS NULL;
CREATE UNIQUE INDEX employee_credentials_employee ON employee_credentials (employee_id) WHERE
  ↳ deleted_at IS NULL;
```

When a received row clashes with a live local row, **the same rule runs on every tablet**, so every tablet ends the same:

- **Two clock-ins for one employee on one day:** keep the day linked to a pay period, otherwise the smaller id. The other day is marked deleted and its sessions move to the kept day (position + 1000), untouched. If the hours overlap, the day is flagged `doublePointage` for a manager. Working time is never deleted.
- **Same supplier linked twice, or two passwords for one employee:** the most recently changed one wins (the larger id on a tie). The other is marked deleted, and the article keeps a default supplier.

These fixes are real changes, written with `SyncQuiet.loud`, so they are sent like any change. They are also logged as `resolved_*` in `sync_errors`, and the sync page shows them under «À vérifier».

7.3 Duplicate names

Two tablets can create “Boissons” offline. Both are accepted. The catalog page shows a notice with a «Fusionner» button (`lib/features/catalog/presentation/widgets/duplicate_notice.dart`), which moves the articles to the copy that holds the most.

Phase 8 Photos (schema v20)

8.1 Photo files have their own queue

Files `lib/data/database/tables/photo_uploads.dart`, `lib/data/database/photo_triggers.dart`

The **row** naming a photo (`items.image_path`, `employees.photo_asset`) travels through the outbox. The **file** travels through `photo_uploads`, also filled by triggers:

```
-- photo set                → ('items', store, file, 'upload')
-- photo replaced/dropped,
-- or its row deleted       → old file 'remove', new file 'upload'
-- one entry per file: set then dropped before sync = a single 'remove'
-- quiet for received rows: their file is downloaded, not uploaded
```

Since v20, employee photos are stored by file name like product photos. The online path is always `<store>/<items|employees>/<file>`.

8.2 Photos in the pass

Files `lib/services/photo_sync.dart`

```
class PhotoSync {
  upload()           // shrink (longest side 1024 px, JPEG) and send queued photos
  removeOld()        // remove replaced photos, only AFTER the row change was sent
  downloadMissing()  // fetch every photo a live row names that this tablet doesn't have
}
```

Order in a pass: upload → push rows → upload again → remove old → pull → download

“Upload again” exists because a photo of a brand-new store is refused until the store itself is on the server.

A photo problem never fails the pass (except being offline). When a photo arrives, the screens refresh (`ProductImages.revision`, `EmployeePhotoStore.revision`).

Known limit A new photo on the server doesn’t trigger a live update on the other tablets. It shows up on their next pass.

Phase 9 The data already on a device

Files lib/data/repositories/local_data_repository.dart

```
enum LocalDataKind { none, demo, own }

kind()          // own = a live store that is not a demo store id
removeDemo()    // real delete of the demo stores (quiet: never meant for a server)
queueEverything() // "touch" every row, parents first → the normal triggers queue it, +
    ↪ every photo
exportBackup(dir) // every synced table to a JSON file, before the account's data replaces it
```

The three cases, at the first account sign-in:

The device holds	What happens
Nothing or only the demo	the demo is wiped, and the account's data arrives with the first sync
Its own data, the account is empty	«Envoyer mes données» / «Utiliser ces données»: demo removed, everything queued and sent
Its own data, the account already has data	a backup JSON file goes to the documents folder, then the account's data replaces it

Screen: /welcome/existing-data (account_existing_data_page.dart).

Phase 10 Screens and messages

Where	What it shows
Sync page, account mode account_sync_view.dart	state in words («À jour», «Synchronisation...», «Hors ligne», «Synchronisation arrêtée»), last sync, changes and photos waiting, restaurant, e-mail, this device's name, «Synchroniser maintenant», the «À vérifier» list with «Compris», and «Se reconnecter» after an expired session
Sync page, demo	an honest notice: the data stays on this device; the demo reset stays here only
Offline banner offline_banner.dart	account mode: the real state («Serveur injoignable · N modifications en attente»); demo mode: the switch that shows offline on demand
Texts	every new text is in lib/l10n/*.arb, with no technical words

```
final isOfflineProvider = Provider<bool>((ref) {
  if (ref.watch(deviceAccessProvider).isAccount) {
    return ref.watch(syncControllerProvider).status == SyncStatus.offline; // the truth
  }
  return ref.watch(offlineModeProvider); // demo switch
});
```

Local schema versions added

Version	What
15	updated_at, deleted_at, store_id on children, touch triggers, stock baseline, PBKDF2
16	outbox table and the generated outbox triggers
17	sync_errors table
18	touch triggers stay quiet during quiet writes
19	“one per” rules become partial unique indexes on live rows
20	photo_uploads table, photo triggers, employee photos stored by file name

Each version has a migration in `app_database.dart` (if `(from < N)`) and a schema dump, and `test/db/migration_test.dart` checks the upgrade keeps the data.

Tests

Where	What	How to run
<code>supabase/tests/database/</code>	49 server tests (pgTAP)	<code>supabase test db</code>
<code>test/db/</code>	499 tests: outbox, stock ledger, push, pull, conflicts, photos, existing data, migrations	<code>flutter test test/db test/password_hash_test.dart</code>
<code>test/*.dart</code>	screen tests: account screens, sync screens, duplicates...	<code>flutter test --concurrency=2 ..</code>
<code>test/integration/</code>	7 tests against a real local Supabase : two tablets, push, pull, conflicts, photos, live updates	<code>flutter test test/integration --concurrency=1 --dart-define-from-file=config/local.json</code>

`test/support/fake_account_backend.dart` is an in-memory server. It can fail on purpose (`failNext`), refuse entries (`rejectWhen`), edit a row while a push is travelling (`duringPush`), or cut a pull halfway (`failPullAfterPages`). Two-tablet tests use two in-memory databases.

If you change something later

Checklist

- **A new column or table to sync:** new drift schema version and dump → `python tool/generate_outbox_triggers.py` → a **new** server migration (never edit an applied one) → add the table to `SyncTables.synced` in the right parent-first position.
- **A new delete:** use `SoftDelete`, never `_db.delete` on a synced table.
- **A write that must not be sent** (seed, recompute): wrap it in `SyncQuiet.run`.
- **Never trust the device clock** to decide a conflict.
- **Never send demo data.**

What is left

Phases 0 and 11: create the cloud “dev” project on `supabase.com`, `supabase link` and `supabase db push`, add a `config/dev.json`, then the manual test on two real devices, an app update over existing data, and a small beta. See “Next step” at the top of `SYNC_PLAN.md`.